

Tema 6:

Listas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Sergio Caverio



Universidad
Rey Juan Carlos

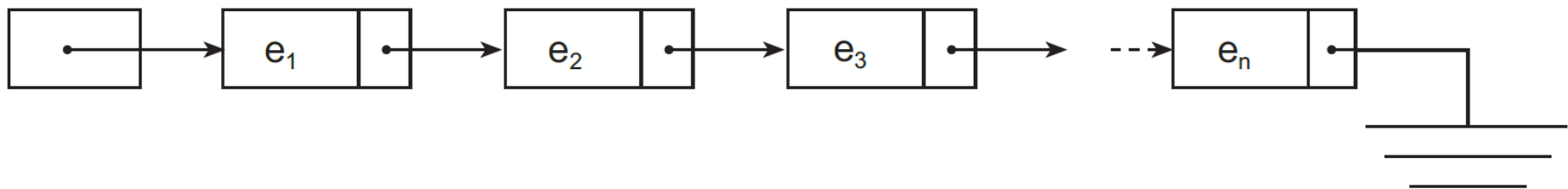
Curso 2025-2026

Contenidos

- Definiciones
- El TAD Lista
- Implementación
 - Lista enlazada simple
 - Lista circular
 - Lista doblemente enlazada
- Listas como base de implementación de otras estructuras de datos

Lista: definiciones

- Estructura de datos lineal, dinámica, que representa una secuencia de elementos, a menudo del mismo tipo.
- Colección o secuencia de elementos dispuestos uno detrás de otro, en la que cada elemento se conecta al siguiente elemento por un “enlace”.



TAD Lista

- Por su flexibilidad, no existe una única definición del TAD lista: debe determinarse primero el tipo de lista que se desea, lo cual condiciona sus operaciones (contrato)
 - Por ejemplo: una lista sin restricciones permite operaciones no toleradas en una lista ordenada, tales como `insertar_final`
- En todo caso, todas comparten operaciones básicas:
 - Consultar si está vacía (`is_empty`)
 - + Inicializar/crear

TAD Lista

- Una abstracción extrema de una lista incluiría las siguientes operaciones (su implementación será diferente en función del tipo de lista):
 - Meter un elemento (**insert**)
 - Eliminar un elemento (**delete**)
 - Consultar si está vacía (**is_empty**)
 - Conocer el primer y último elemento (**first/last**)
 - Comprobar si un elemento está (**in**)
 - + Inicializar/crear (**initialize**)

TAD Lista: especificación

- **Nombre:** Lista.
- **Valores posibles:** *vacía, otros (según elementos).*
- **Operaciones:**
 - `insert_at_end`
 - `insert_at_begin`
 - `delete_at_end`
 - `delete_at_begin`
 - `first`
 - `last`
 - `is_empty`
 - `in_list`
- **Uso de otros tipos:** tipo base, punteros



Implementación y tipos

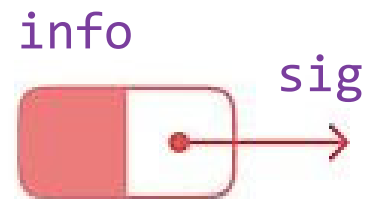
- La implementación de listas en estructuras de soporte estáticas como arrays no es habitual: Siempre en memoria dinámica
- Tipos (por implementación):
 - Lista enlazada
 - Lista doblemente enlazada
 - Lista circular
- Otros tipos (por lógica interna): Lista ordenada

Lista enlazada simple

Implementación con memoria dinámica

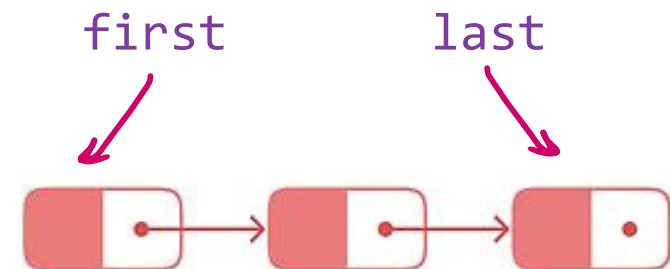
- Utilizamos una estructura nodo

```
type
  nodo = record
    info: integer;
    sig: ^nodo;
  end;
```



- Y luego una estructura con los dos manejadores de la lista:

```
tListaSimple = record
  first, last: ^nodo;
end;
```



Lista enlazada: implementación

{Operaciones básicas}

```
procedure initialize(var list: tListaSimple);  
function is_empty(list: tListaSimple): boolean;  
function first(list: tListaSimple): integer;  
function last(list: tListaSimple): integer;  
procedure insert_at_end(var list: tListaSimple; x: integer);  
procedure insert_at_begin(var list: tListaSimple; x: integer);  
procedure delete_at_end(var list: tListaSimple);  
procedure delete_at_begin(var list: tListaSimple);  
function in_list(list: tListaSimple; x: integer): boolean;
```

Lista enlazada: implementación

{Otras operaciones}

```
function to_string(list: tListaSimple): string;  
procedure clear(var list: tListaSimple);  
function num_elems(list: tListaSimple): integer;  
procedure copy(list: tListaSimple; var c2: tListaSimple);
```



Lista enlazada: implementación

implementation

```
procedure initialize(var list: tListaSimple);  
begin  
    list.first := nil;  
    list.last := nil;  
end;
```

```
function is_empty(list: tListaSimple): boolean;  
begin  
    is_empty := list.first = nil;  
end;
```



Lista enlazada: implementación

```
function first(list: tListaSimple): integer;  
begin  
    if not is_empty(list) then  
        first := list.first^.info;  
    end;
```

```
function last(list: tListaSimple): integer;  
begin  
    if not is_empty(list) then  
        last := list.last^.info;  
    end;
```



Lista enlazada: implementación

```
procedure insert_at_end(var list: tListaSimple; x: integer);  
var  
    newNode: ^nodo;  
begin  
    new(newNode);  
    newNode^.info := x;  
    newNode^.sig := nil;  
    if is_empty(list) then  
        list.first := newNode  
    else  
        list.last^.sig := newNode;  
        list.last := newNode;  
    end;  
end;
```

Lista enlazada: implementación

```
procedure insert_at_begin(var list: tListaSimple; x: integer);  
var  
    newNode: ^nodo;  
begin  
    new(newNode);  
    newNode^.info := x;  
    newNode^.sig := list.first;  
    list.first := newNode;  
    if list.last = nil then  
        list.last := newNode;  
end;
```



Lista enlazada: implementación

```
procedure delete_at_begin(var list: tListaSimple);  
var  
    temp: ^nodo;  
begin  
    if not is_empty(list) then begin  
        temp := list.first;  
        list.first := list.first^.sig;  
        dispose(temp);  
        if list.first = nil then  
            list.last := nil;  
        end;  
    end;  
end;
```



Lista enlazada: implementación

```
procedure delete_at_end(var list: tListaSimple);
var
    current, prev: ^nodo;
begin
    if not is_empty(list) then begin
        current := list.first;
        prev := nil;
        while current^.sig <> nil do begin
            prev := current;
            current := current^.sig;
        end;
        if prev = nil then begin // Solo hay un elemento en la lista
            dispose(list.first);
            list.first := nil;
            list.last := nil;
        end
        else begin // Hay más de un elemento en la lista
            dispose(current);
            prev^.sig := nil;
            list.last := prev;
        end;
    end;
end;
```

Lista enlazada: implementación

```
procedure delete(var list: tListaSimple; x: integer);
var
    current, prev: ^nodo;
begin
    current := list.first;
    prev := nil;
    while (current <> nil) and (current^.info <> x) do begin
        // Buscar el nodo
        prev := current;
        current := current^.sig;
    end;
    if current <> nil then begin // Si se encontró el nodo y lista no vacía
        if prev = nil then // Si el nodo a eliminar es el primero
            list.first := current^.sig;
        else // Si el nodo a eliminar no es el primero
            prev^.sig := current^.sig;
        if current = list.last then // Si el nodo a eliminar es el último
            list.last := prev;
        dispose(current); // Libera el nodo
    end;
end;
```

Lista enlazada: implementación

```
function in_list(list: tListaSimple; x: integer): boolean;  
var  
    current: ^nodo;  
begin  
    current := list.first;  
    while (current <> nil) and (current^.info <> x) do  
        current := current^.sig;  
    in_list := current <> nil;  
end;
```



Lista enlazada: implementación

// Implementación recursiva

```
function rec_in_list(list: tListaSimple; x: integer): boolean;  
var sub_list : tListaSimple;  
begin  
    if list.first = nil then  
        rec_in_list := false // No se encontró el elemento  
    else if list.first^.info = x then // Se encontró el elemento  
        rec_in_list := true  
    else begin  
        sub_list.first := list.first^.sig; // Comienza en el sig. nodo  
        sub_list.last := list.last; // Termina en el mismo nodo  
        rec_in_list := rec_in_list(sub_list, x); // Llamada recursiva  
    end;  
end;
```



Lista enlazada: implementación

```
function to_string(list: tListaSimple): string;
var
    current: ^nodo;
    str: string;
begin
    str := '';
    current := list.first;
    while current <> nil do
    begin
        str := str + IntToStr(current^.info) + ' ';
        current := current^.sig;
    end;
    to_string := str;
end;
```



Lista enlazada: implementación

```
procedure clear(var list: tListaSimple);  
var  
    temp: ^nodo;  
begin  
    while list.first <> nil do  
    begin  
        temp := list.first;  
        list.first := list.first^.sig;  
        dispose(temp); // Libera cada nodo  
    end;  
    list.last := nil; // La lista está vacía  
end;
```



Lista enlazada: implementación

```
function num_elems(list: tListaSimple): integer;  
var  
    current: ^nodo;  
    count: integer;  
begin  
    count := 0;  
    current := list.first;  
    while current <> nil do begin  
        count := count + 1;  
        current := current^.sig;  
    end;  
    num_elems := count;  
end;
```



Lista enlazada: implementación

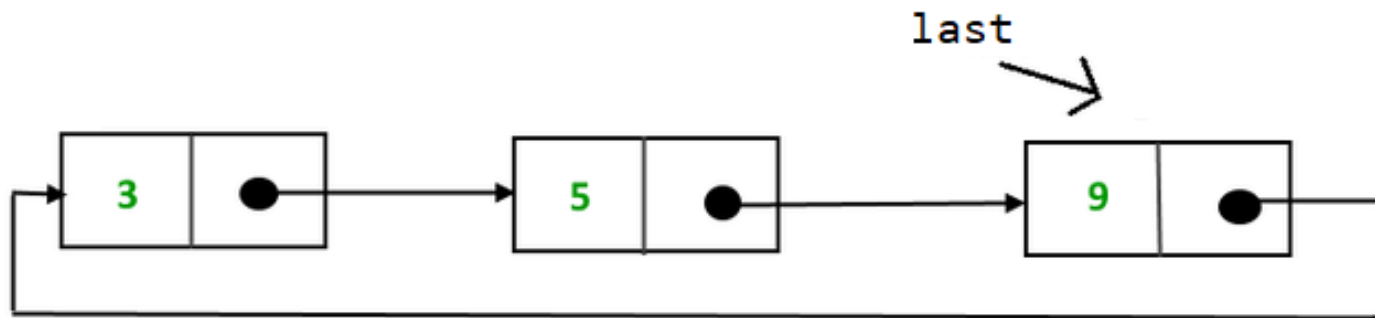
```
procedure copy(list: tListaSimple; var list2: tListaSimple);  
var  
    current: ^nodo;  
begin  
    initialize(list2);  
    current := list.first;  
    while current <> nil do  
        begin  
            insert_at_end(list2, current^.info);  
            current := current^.sig;  
        end;  
    end;  
end;
```



Lista circular

Lista circular

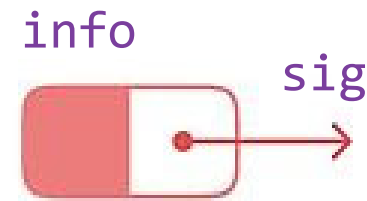
- El último nodo contiene un puntero al primero, de modo que:
 - La lista circular no tiene principio ni fin.
 - No hay ningún **NIL** en la parte **sig** de ninguno de los nodos.
- Se gestiona con un solo manejador, apuntando al último elemento de la lista
- El primero se consigue como el siguiente del último



Lista circular

- La estructura de manejo es un único puntero, teniendo el nodo la misma estructura que en una lista simple:

```
type
  nodo = record
    info: integer;
    sig: ^nodo;
  end;
```



- Para manejar la lista, bastará con un manejador:

```
tListaCircular = record
  last: ^nodo;
end;
```

Lista circular

- Sus operaciones pueden variar según el uso que se le quiera dar
- Normalmente da igual guardar el orden de ingreso, por lo que tras una operación de inserción o borrado el puntero manejador podría cambiar

{Operaciones básicas}

```
procedure initialize(var list: tListaCircular);  
function is_empty(list: tListaCircular): boolean;  
function first(list: tListaCircular): integer;  
function last(list: tListaCircular): integer;  
procedure insert_at_end(var list: tListaCircular; x: integer);  
procedure insert_at_begin(var list: tListaCircular; x: integer);  
procedure delete(var list: tListaCircular; x: integer);  
function in_list(list: tListaCircular; x: integer): boolean;
```

Lista circular

{Otras operaciones}

```
function to_string(list: tListaCircular): string;  
procedure clear(var list: tListaCircular);  
function num_elems(list: tListaCircular): integer;  
procedure copy(list: tListaCircular; var c2: tListaCircular);
```

Lista circular

```
procedure initialize(var list: tListaCircular);  
begin  
    list.last := nil;  
end;
```

```
function is_empty(list: tListaCircular): boolean;  
begin  
    is_empty := list.last = nil;  
end;
```

Lista circular

```
function last(list: tListaCircular): integer;  
begin  
    if not is_empty(list) then  
        last := list.last^.info;  
    end;
```

```
function first(list: tListaCircular): integer;  
begin  
    if not is_empty(list) then  
        first := list.last^.sig^.info;  
    end;
```

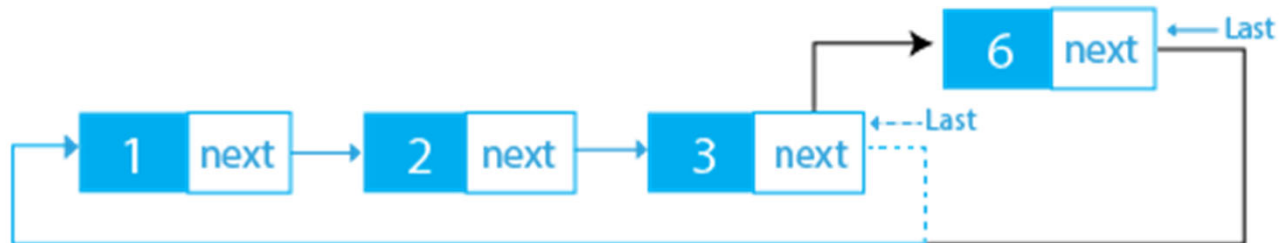
Lista circular

```
procedure insert_at_begin(var list: tListaCircular; x: integer);  
var  
    aux: ^nodo;  
begin  
    new(aux);  
    aux^.info := x;  
    if is_empty(list) then begin  
        aux^.sig := aux;  
        list.last := aux;  
    end  
    else begin  
        aux^.sig := list.last^.sig;  
        list.last^.sig := aux;  
    end;  
end;  
end;
```



Lista circular

```
procedure insert_at_end(var list: tListaCircular; x: integer);  
var  
    aux: ^nodo;  
begin  
    new(aux);  
    aux^.info := x;  
    if is_empty(list) then  
        aux^.sig := aux  
    else begin  
        aux^.sig := list.last^.sig;  
        list.last^.sig := aux;  
    end;  
    list.last := aux;  
end;
```



Lista circular: delete

1. Punteros necesarios

Para eliminar un nodo en una lista enlazada (simple o circular), necesitamos tener acceso a:

- **current**: El nodo que estamos analizando (y que potencialmente vamos a borrar).
- **prev**: El nodo **inmediatamente anterior**, porque al borrar **current**, necesitaremos que **prev** pase a enlazarse con el siguiente nodo (`prev^.sig := current^.sig`).

Nota clave en Circulares: En una lista circular, el "anterior" al primer nodo es el último nodo (**last**). Esto es crucial para inicializar correctamente nuestros punteros de búsqueda.

Lista circular: delete

2. Casos a considerar (Lógica de Borrado)

No podemos aplicar la misma lógica siempre. Debemos dividir el problema en escenarios:

1. **Lista Vacía:** No hay nada que borrar. Terminamos.
2. **Lista No Vacía**
 - **2a: Hay un único nodo** (`list.last^.sig = list.last`)
 - Si ese nodo es el que buscamos → **dispose** y ponemos `list.last` a **nil**.
 - **2b: Hay más de un nodo**
 - Inicializamos punteros: **prev** empieza en `list.last` y **current** en el primero.
 - **Buscar:** Recorremos la lista. ¡Cuidado con no entrar en un bucle infinito! (parar si volvemos al principio).

Lista circular: delete

2. Casos a considerar (cont.)

- **2b: Hay más de un nodo**
 - Inicializamos punteros: **prev** empieza en **list.last** y **current** en el primero.
 - **Buscar:** Recorremos la lista. ¡Cuidado con no entrar en un bucle infinito! (parar si volvemos al principio).
 - **Si lo encontramos:**
 - “Punteamos” el nodo: **prev^.sig := current^.sig**
 - **Caso especial:** Si el nodo borrado era el último (**last**), debemos actualizar **list.last** para que apunte al anterior (**prev**).
 - Liberamos memoria (**dispose**).

Lista circular: delete

Analicemos el código,
caso por caso.

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```

Lista circular: delete

CASO 1: Lista Vacía

```
if not is_empty(list) then
```

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```

Lista circular: delete

CASO 2: Lista NO Vacía

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```

Lista circular: delete

CASO 2a: Hay un único nodo

Y ese nodo es el que buscamos

```
if list.last^.sig = list.last then
begin
    if list.last^.info = x then
    begin
        dispose(list.last);
        list.last := nil;
    end;
end
```

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true;
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

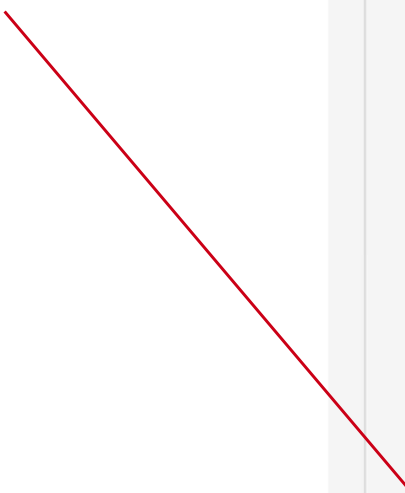
            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```


Lista circular: delete

CASO 2b: Hay más de un nodo



```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```

Lista circular: delete

CASO 2b: Hay más de un nodo

- Inicializamos punteros: **prev** empieza en **list.last** y **current** en el primero.
- **Buscar:** Recorremos la lista. ¡Cuidado con no entrar en un bucle infinito! (parar si volvemos al principio).

```
found := false;
prev := list.last;
current := list.last^.sig;

repeat
    if current^.info = x then
        found := true
    else
        begin
            prev := current;
            current := current^.sig;
        end;
until found or (current = list.last^.sig);
```

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
        begin
            if list.last^.sig = list.last then
                begin
                    if list.last^.info = x then
                        begin
                            dispose(list.last);
                            list.last := nil;
                        end;
                    end
                else
                    begin
                        found := false;
                        prev := list.last;
                        current := list.last^.sig;

                        repeat
                            if current^.info = x then
                                found := true
                            else
                                begin
                                    prev := current;
                                    current := current^.sig;
                                end;
                        until found or (current = list.last^.sig);

                        if found then
                            begin
                                prev^.sig := current^.sig;

                                if current = list.last then
                                    list.last := prev;
                                dispose(current);
                            end;
                        end;
                    end;
                end;
            end;
        end;
```

Lista circular: delete

CASO 2b: Hay más de un nodo

- Inicializamos punteros: **prev** empieza en **list.last** y **current** en el primero.
- **Buscar:** Recorremos la lista. ¡Cuidado con no entrar en un bucle infinito! (parar si volvemos al principio).

Se puede hacer con un While en vez de un repeat-until pero necesitaríamos una comprobación extra para el último nodo.

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```

Lista circular: delete

CASO 2b: Hay más de un nodo

- Si lo encontramos:
 - Desenlazamos el nodo: $\text{prev}^{\wedge}.\text{sig} := \text{current}^{\wedge}.\text{sig}$
 - **Caso Especial:** Si el nodo borrado era el último (**last**), debemos actualizar **list.last** para que apunte al anterior (**prev**).
 - Liberamos memoria (**dispose**).

```
if found then
begin
    prev^.sig := current^.sig;

    if current = list.last then
        list.last := prev;

    dispose(current);
end;
```

```
procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    if not is_empty(list) then
    begin
        if list.last^.sig = list.last then
        begin
            if list.last^.info = x then
            begin
                dispose(list.last);
                list.last := nil;
            end;
        end
        else
        begin
            found := false;
            prev := list.last;
            current := list.last^.sig;

            repeat
                if current^.info = x then
                    found := true
                else
                begin
                    prev := current;
                    current := current^.sig;
                end;
            until found or (current = list.last^.sig);

            if found then
            begin
                prev^.sig := current^.sig;

                if current = list.last then
                    list.last := prev;

                dispose(current);
            end;
        end;
    end;
end;
```

Lista circular: delete

¿Es esta la única manera de hacerlo?: NO

El código que acabamos de ver es una solución **correcta y válida**, pero no es la única. En programación, y especialmente trabajando con punteros, hay muchas formas de llegar al mismo resultado.

Consejos:

- **No memorices el código.** Aprenderse de memoria cada línea de `prev^.sig := current^.sig` es inútil si no visualizas lo que ocurre.
- **Entiende la lógica de punteros.** Dibuja la lista en un papel. Mueve las flechas (punteros). Entiende *por qué* necesitamos un puntero `prev` o *por qué* el `last` es especial.
- **Haz tu propia implementación.** Si entiendes el concepto (buscar → desenlazar → liberar), podrás escribir tu propio código `delete` sin necesidad de copiar este.

La memoria falla, la lógica permanece.

Lista circular

```
function in_list(list: tListaCircular; x: integer): boolean;
var
    aux: ^nodo;
    found: boolean;
begin
    found := false;
    if not is_empty(list) then begin
        aux := list.last^.sig;
        while (aux <> list.last) and not found do begin
            // busca el elemento
            if aux^.info = x then found := true
            else aux := aux^.sig;
        end;
        if aux^.info = x then found := true;
    end;
    in_list := found;
end;
```

Lista circular

```
function to_string(list: tListaCircular): string;
var
    aux: ^nodo;
    s: string;
begin
    s := '';
    if not is_empty(list) then begin
        aux := list.last^.sig;
        repeat
            s := s + IntToStr(aux^.info) + ' ';
            aux := aux^.sig;
        until aux = list.last^.sig;
    end;
    to_string := s;
end;
```



Lista circular

```
procedure clear(var list: tListaCircular);  
var  
    aux: ^nodo;  
begin  
    if not is_empty(list) then begin  
        aux := list.last^.sig;  
        repeat  
            list.last^.sig := aux^.sig;  
            dispose(aux);  
            aux := list.last^.sig;  
        until aux = list.last^.sig;  
        list.last := nil;  
    end;  
end;
```



Lista circular

```
function num_elems(list: tListaCircular): integer;  
var  
    aux: ^nodo;  
    count: integer;  
begin  
    count := 0;  
    if not is_empty(list) then begin  
        aux := list.last;  
        repeat  
            count := count + 1;  
            aux := aux^.sig;  
        until aux = list.last;  
    end;  
    num_elems := count;  
end;
```



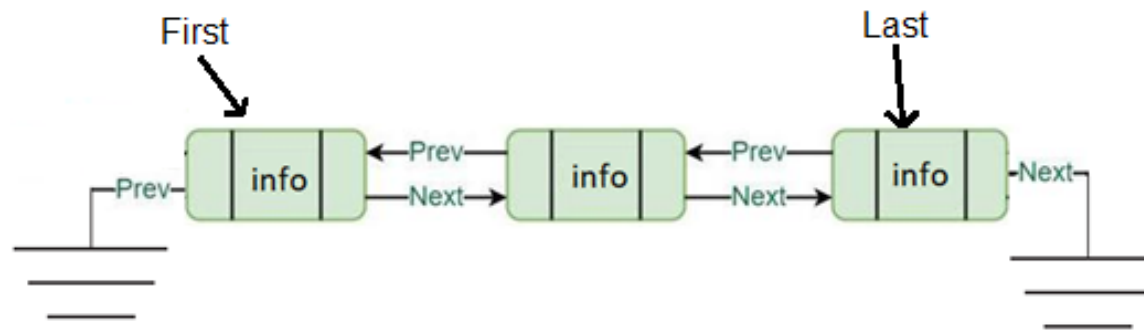
Lista circular

```
procedure copy(list: tListaCircular; var list2: tListaCircular);  
var  
    aux, new_node: ^nodo;  
begin  
    initialize(list2);  
    if not is_empty(list) then begin  
        aux := list.last;  
        repeat  
            insert_at_end(list2, aux^.info);  
            aux := aux^.sig;  
        until aux = list.last;  
    end;  
end;
```

Lista doblemente enlazada

Lista doblemente enlazada

- Lista doblemente enlazada



- Ventajas:
 - Facilita inserciones y borrados en ambos extremos (y en general en cualquier lugar de la lista)
 - Simplifica recorridos en ambos sentidos
 - Ideal para la implementación de estructuras tales como grafos, diccionarios y otras

Lista doblemente enlazada

type

nodo = record

info: integer; // Información almacenada en el nodo

sig: ^nodo; // Puntero al siguiente nodo

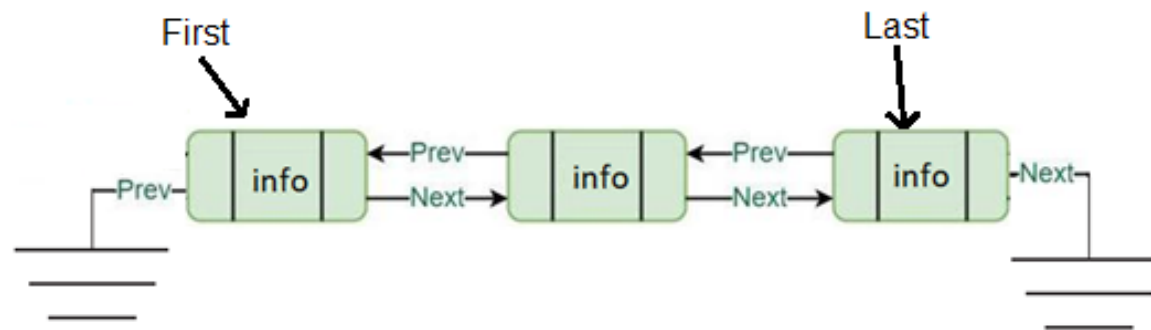
ant: ^nodo; // Puntero al nodo anterior

end;

tListaDoble = record

first, last: ^nodo; // Punteros al primer y último nodo

end;



Lista doblemente enlazada

{Operaciones básicas}

```
procedure initialize(var list: tListaDoble);  
function is_empty(list: tListaDoble): boolean;  
function first(list: tListaDoble): integer;  
function last(list: tListaDoble): integer;  
procedure insert_at_end(var list: tListaDoble; x: integer);  
procedure insert_at_begin(var list: tListaDoble; x: integer);  
procedure delete_at_end(var list: tListaDoble);  
procedure delete_at_begin(var list: tListaDoble);  
function in_list(list: tListaDoble; x: integer): boolean;
```

{Otras operaciones}

```
function to_string(list: tListaDoble): string;  
procedure clear(var list: tListaDoble);  
function num_elems(list: tListaDoble): integer;  
procedure copy(list: tListaDoble; var c2: tListaDoble);
```



Lista doblemente enlazada

```
procedure initialize(var list: tListaDoble);  
begin  
    list.first := nil;  
    list.last := nil;  
end;  
  
function is_empty(list: tListaDoble): boolean;  
begin  
    is_empty := list.first = nil;  
end;
```



Lista doblemente enlazada

```
function first(list: tListaDoble): integer;  
begin  
    if not is_empty(list) then  
        first := list.first^.info;  
    end;
```

```
function last(list: tListaDoble): integer;  
begin  
    if not is_empty(list) then  
        last := list.last^.info;  
    end;
```



Lista doblemente enlazada

```
procedure insert_at_end(var list: tListaDoble; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    aux^.sig := nil;
    aux^.ant := list.last;
    if list.last=nil then
        list.first := aux
    else
        list.last^.sig := aux;
    list.last := aux;
end;
```



Lista doblemente enlazada

```
procedure insert_at_begin(var list: tListaDoble; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    aux^.sig := list.first;
    aux^.ant := nil;
    if list.first = nil then
        list.last := aux
    else
        list.first^.ant := aux;
    list.first := aux;
end;
```



Lista doblemente enlazada

```
procedure delete_at_end(var list: tListaDoble);
var
    aux: ^nodo;
begin
    if not is_empty(list) then
    begin
        aux := list.last;
        list.last := aux^.ant;
        if list.last <> nil then
            list.last^.sig := nil
        else
            list.first := nil;
        dispose(aux);
    end;
end;
```



Lista doblemente enlazada

```
procedure delete_at_begin(var list: tListaDoble);
var
    aux: ^nodo;
begin
    if not is_empty(list) then
    begin
        aux := list.first;
        list.first := aux^.sig;
        if list.first <> nil then
            list.first^.ant := nil
        else
            list.last := nil;
        dispose(aux);
    end;
end;
```



Lista doblemente enlazada

```
function in_list(list: tListaDoble; x: integer): boolean;  
var  
    aux: ^nodo;  
    found: boolean;  
begin  
    aux := list.first;  
    found := false;  
    while (aux <> nil) and not found do  
        if aux^.info = x then found := true;  
        else aux := aux^.sig;  
    in_list := found;  
end;
```



Lista doblemente enlazada

```
function to_string(list: tListaDoble): string;  
var  
    aux: ^nodo;  
    s: string;  
begin  
    aux := list.first;  
    s := '';  
    while aux <> nil do begin  
        s := s + IntToStr(aux^.info) + ' ';  
        aux := aux^.sig;  
    end;  
    to_string := s;  
end;
```



Lista doblemente enlazada

```
procedure clear(var list: tListaDoble);  
begin  
    while not is_empty(list) do  
        delete_at_begin(list);  
    end;
```

Lista doblemente enlazada

```
function num_elems(list: tListaDoble): integer;  
var  
    aux: ^nodo;  
    count: integer;  
begin  
    aux := list.first;  
    count := 0;  
    while aux <> nil do begin  
        count := count + 1;  
        aux := aux^.sig;  
    end;  
    num_elems := count;  
end;
```



Lista doblemente enlazada

```
procedure copy(list: tListaDoble; var list2: tListaDoble);  
var  
    aux: ^nodo;  
begin  
    initialize(list2);  
    aux := list.first;  
    while aux <> nil do begin  
        insert_at_begin(list2, aux^.info);  
        aux := aux^.sig;  
    end;  
end;
```



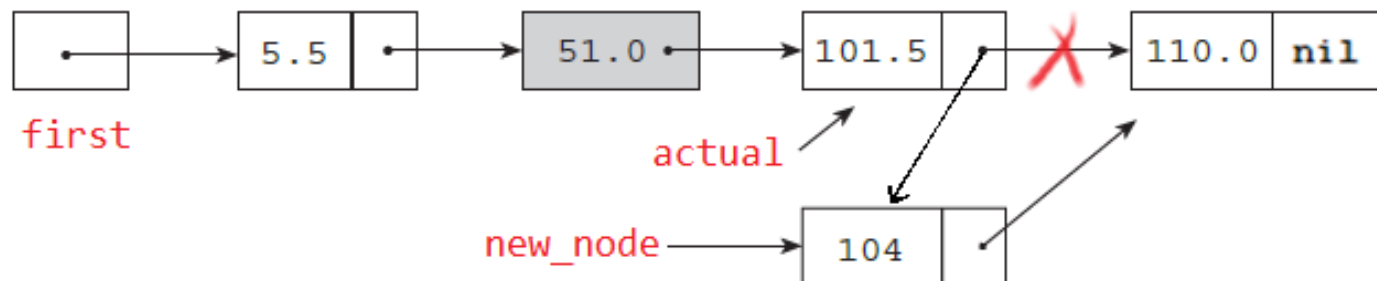
Listas como base para la implementación de otras estructuras

Base para estructuras más complejas

- La flexibilidad de las listas, tanto enlazadas como doblemente enlazadas, permite la implementación de estructuras con restricciones adicionales, o más complejas tales como:
 - Listas ordenadas
 - Colas de prioridades
 - Colas doblemente terminadas (*deques*)
 - Mapas / Diccionarios
 - Grafos

Lista ordenada

- Estructura de datos dinámica, cuyos elementos se encuentran en todo momento ordenados.
- La forma de insertar un elemento en una lista ordenada siempre realiza la operación de tal forma que la lista resultante mantiene la propiedad de ordenación
 - Para ello, en primer lugar, determina la posición de inserción y, a continuación, ajusta los enlaces.



Lista ordenada

```
procedure insert(var list: tSortedList; x: integer);
var
    new_node, actual: ^nodo;
begin
    new(new_node);
    new_node^.info := x;
    if is_empty(list) or (x < list.first^.info) then // insertar el primero
    begin
        new_node^.next := list.first;
        list.first := new_node;
    end
    else
    begin // Situar actual en el nodo anterior a la inserción
        actual := list.first;
        while (actual^.next <> nil) and (x > actual^.next^.info) do
            actual := actual^.next;
        end;
        new_node^.next := actual^.next;
        actual^.next := new_node;
    end;
end;
```

Cola de prioridades

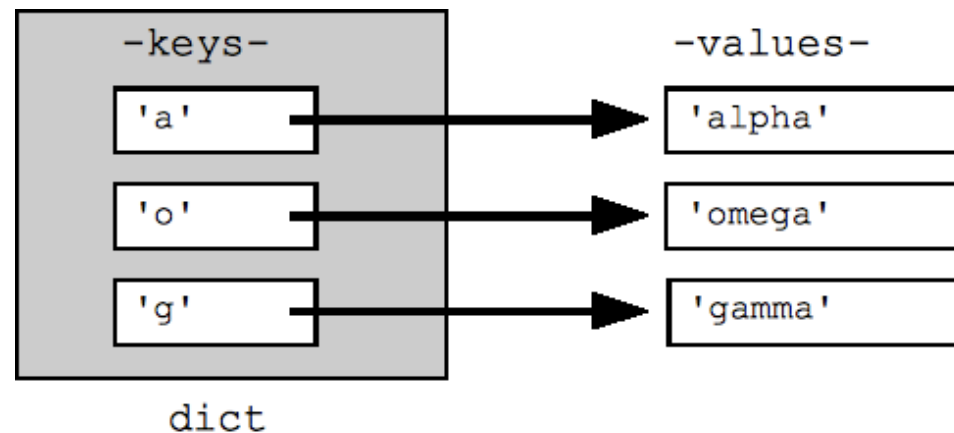
- Cada elemento de la cola lleva una prioridad asociada.
- Los componentes de la cola se ordenan según su prioridad.
- Hay 2 alternativas de implementación:
 - Añadir un campo a cada nodo con su prioridad, manteniendo la cola ordenada por un doble criterio: prioridad y a igual prioridad FIFO.
 - Crear tantas colas como prioridades haya, y almacenar cada elemento en su cola.
- Habitualmente si dos elementos tienen la misma prioridad se sirve primero al que lleva más tiempo esperando en la cola.

Cola doblemente terminada (*deque*)

- Una *deque* (pronunciado “DEK”) es una cola que permite inserciones en ambos extremos y no solo en el final.
- Útil en aplicaciones donde por algún motivo uno de los elementos debe entrar en un nivel prioritario (al inicio de la cola)
 - Un restaurante donde un cliente VIP “se cuela”
 - Cuando el elemento saliente no ha podido ser servido y se reinserta en la primera posición para no esperar de nuevo toda la cola.
 - etc.
- Una implementación obvia es una limitación de la lista general.

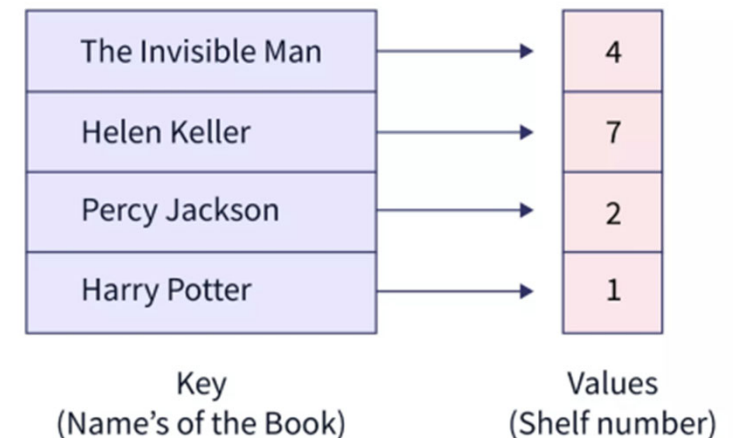
Mapa o diccionario

- Un mapa (C++), hashmap (Java) o diccionario (Python) permite guardar valores asociados a claves únicas.
- Está compuesto por parejas clave-valor, tales como:
 - URL / IP
 - Username / password
 - Palabra / frecuencia con que aparece en un texto
 - Etc.



Mapas: implementaciones

- Primera aproximación: una **lista no ordenada** en la que se inserta siempre al final.
- Al buscar una clave a menudo hay que recorrer la lista completa
 - Inserción: $O(1)$
 - Búsqueda: $O(n)$
- La búsqueda por clave es crítica: en inserción, en acceso y en borrado, lo que lleva a buscar otras implementaciones.



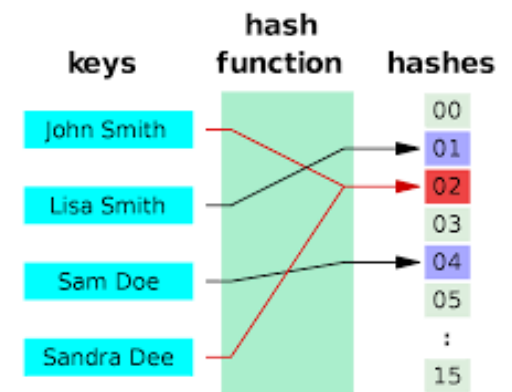
Mapas: implementaciones

- Una **lista ordenada de claves** mejora las búsquedas a costa de perjudicar la eficiencia de las inserciones.
- Las búsquedas son $O(n)$ pero el caso medio mejora
- Las inserciones pasan a $O(n)$ aunque son menos frecuentes y por ende puede compensar.

keys	2	5	6	8	
values	b	z	p	m	

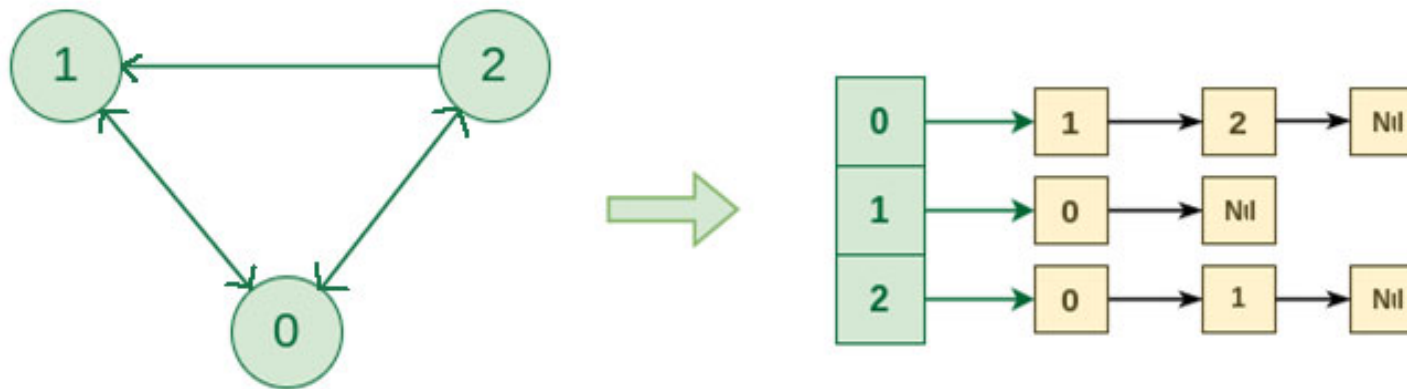
Mapas: implementaciones

- Para conseguir búsquedas en $O(1)$ es posible utilizar un **array de enteros** para las claves.
- Necesita:
 - Claves enteras
 - Si las claves no son enteros $\rightarrow$ función hash convierte una clave k en un entero
 - Función de compresión garantiza que todo hash esté en el rango válido de índices $\rightarrow$ evita limitaciones por el tamaño del array
- Habrá **colisiones**: varias claves dan el mismo índice en el array tras aplicarles la función hash



Grafos

- Un grafo puede implementarse mediante listas de listas:
 - Una lista de nodos
 - Una lista de nodos adyacentes a un cierto nodo (listas de adyacencia)



(más sobre esto, pronto)